39

插叙：文件和目录

到目前为止，我们已经看到两个关键的操作系统抽象（abstraction）是如何发展起来的：一个是进程（process），它是对 CPU 的虚拟化（virtualization）；另一个是地址空间（address space），它是对内存的虚拟化。这两个抽象配合起来，让程序运行时仿佛身处一个私有的、隔离（isolated）的世界：仿佛有自己的处理器（一个或多个），仿佛有自己的内存。这种假象让系统编程（systems programming）容易得多，因此如今不仅在台式机和服务器上随处可见，在手机之类的各种可编程平台上也越来越普遍。

在这一部分，我们给虚拟化这幅拼图再添上关键的一块：**持久存储**（persistent storage）。持久存储设备（persistent storage device），比如经典的**硬盘驱动器**（hard disk drive）或更现代的**固态存储设备**（solid-state storage device），会永久地（至少是长期地）保存信息。内存的内容在断电（power loss）后就会丢失，持久存储设备（persistent storage device）则能完好地保存数据。因此，操作系统对这类设备必须格外小心：用户真正在乎的数据就放在这里。

关键问题：如何管理持久存储设备
操作系统应该如何管理持久存储设备（persistent storage device）？有哪些 API？实现中有哪些重要的方面？

因此，在接下来的几章里，我们将探讨管理持久数据的关键技术，重点是提高性能和可靠性（reliability）的方法。不过，我们先从 API 讲起，概要介绍一下和 UNIX 文件系统（file system）打交道时会用到的接口。

39.1 文件和目录

随着时间的推移，存储虚拟化（storage virtualization）形成了两个关键的抽象。第一个是**文件**（file）。文件就是一个线性的字节数组，其中每个字节都可以读写。每个文件都有某种 **low-level name**，通常是一个数字；用户往往察觉不到这个名称（后面会看到）。由于历史原因，文件的 low-level name 常被称为它的 **inode 号**（inode number，也叫 **i-number**）。后面几章我们会深入学习 inode；现在只要假设每个文件都关联着一个 inode 号就行。

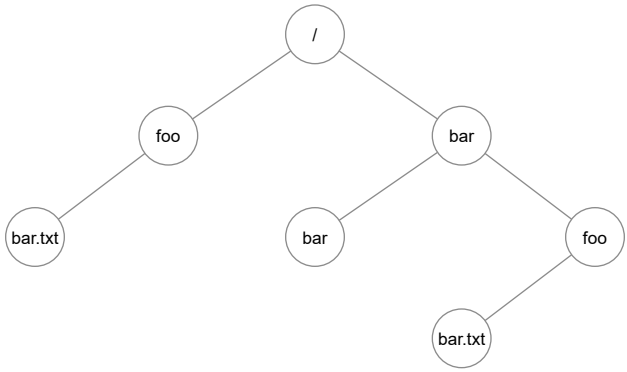


图 39.1：**一棵目录树示例**

在大多数系统中，操作系统并不太了解文件的结构（比如它是一张图片、一个文本文件，还是 C 代码）；文件系统的职责只是把这些数据持久地存放在磁盘上，并确保你再次请求这些数据时，拿到的就是当初放进去的东西。做到这一点可没有看上去那么简单！

第二个抽象是**目录**（directory）。目录和文件一样，也有一个 low-level name，即 inode 号，但它的内容非常特殊：它包含一个由（用户可读的名称，low-level name）对组成的列表。例如，假设有一个 low-level name 为“10”的文件，用户用“foo”这个可读的名称来称呼它。那么“foo”所在的目录中就会有一个条目（“foo”，“10”），把用户可读的名称映射到 low-level name。目录中的每个条目指向的要么是文件，要么是其他目录。把目录放进其他目录，用户就能构建任意的**目录树**（directory tree，也叫**目录层次结构**，directory hierarchy），所有文件和目录都存放在这棵树下。

目录层次结构（directory hierarchy）从**根目录**（root directory）开始（在基于 UNIX 的系统中，根目录就记作 `/`），并用某种**分隔符**（separator）依次命名后面的**子目录**（sub-directories），直到命名到想要的文件或目录为止。例如，如果用户在根目录 `/` 中创建了一个目录 `foo`，然后又在目录 `foo` 中创建了一个文件 `bar.txt`，我们就可以用这个文件的**绝对路径名**（absolute pathname）来指代它，在这个例子里就是 `/foo/bar.txt`。图 39.1 是一棵更复杂的目录树；例子中有效的目录有 `/`、`/foo`、`/bar`、`/bar/bar`、`/bar/foo`，有效的文件有 `/foo/bar.txt` 和 `/bar/foo/bar.txt`。

提示：仔细考虑命名

命名是计算机系统的一个重要方面 [SK09]。在 UNIX 系统中，你能想到的几乎一切都由文件系统来命名。不仅是文件，设备、管道（pipe），甚至进程 [K84]，都能在一个看上去平平无奇的文件系统中找到。这种统一的命名方式简化了你对系统的概念模型，也让系统更简单、更模块化（modular）。所以，无论什么时候创建系统或接口，都要仔细考虑你用的是什么名字。

 © 2008–23, ARPACI-DUSSEAU

只要位于文件系统树的不同位置，目录和文件就可以重名（例如，图中有两个名为 `bar.txt` 的文件：`/foo/bar.txt` 和 `/bar/foo/bar.txt`）。

你可能还注意到，这个例子中的文件名通常由两部分组成：`bar` 和 `txt`，中间用句点隔开。第一部分是任意的名字，而文件名的第二部分通常用来表示文件的**类型**（type），比如它是 C 代码（例如 `.c`）、图片（例如 `.jpg`）还是音乐文件（例如 `.mp3`）。不过，这通常只是一种**约定**（convention）：一般并没有什么机制强制要求名为 `main.c` 的文件里装的一定是 C 源代码。

由此可见，文件系统提供了一项很棒的功能：一种为我们感兴趣的所有文件**命名**（name）的便捷方式。名字在系统中很重要，因为访问任何资源的第一步，就是能够给它命名。因此，在 UNIX 系统中，文件系统提供了一种统一的方式，来访问磁盘、U 盘、CD-ROM 以及许多其他设备上的文件，而且还能访问许多别的东西，它们全都位于同一棵目录树之下。

39.2 文件系统接口

现在我们来更详细地讨论文件系统接口。先从创建、访问和删除文件这些基础操作讲起。你可能觉得这很简单，但在这个过程中，我们会遇到一个用来删除文件的神秘调用：`unlink()`。希望读完本章，你就不觉得这个谜有多神秘了！

39.3 创建文件

我们从最基本的操作开始：创建文件。这要用 `open` 系统调用（system call）：调用 `open()` 并传入 `O_CREAT` 标志，程序就能创建一个新文件。下面的示例代码在当前工作目录（current working directory）中创建一个名为“foo”的文件：

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC,
              S_IRUSR|S_IWUSR);
```

例程 `open()` 可以接受许多不同的标志。在这个例子中，第二个参数表示：如果文件不存在就创建它（`O_CREAT`）；确保文件只能写入（`O_WRONLY`）；如果文件已经存在，就把

补充：`creat()` 系统调用

创建文件的老办法是调用 `creat()`，如下所示：

```
// 可选：再加一个参数来设置权限
int fd = creat("foo");
```

你可以把 `creat()` 看作带有以下标志的 `open()`：

```
O_CREAT | O_WRONLY | O_TRUNC
```

由于 `open()` 就能创建文件，`creat()` 已经不太常用了（其实它完全可以实现为一个调用 `open()` 的库函数）；不过，它在 UNIX 的传说里确实占有一席之地。具体来说，有人问 Ken Thompson，如果重新设计 UNIX，他会在哪些地方换种做法，他回答说：“我会给 creat 加上那个 e。”

它截断为零字节，从而删除原有的全部内容（O_TRUNC）。第三个参数指定权限，这里是让文件的属主（owner）可读可写。

open() 的一个重要方面是它的返回值：**文件描述符**（file descriptor）。文件描述符就是一个整数，每个进程私有，UNIX 系统用它来访问文件；因此，文件一旦打开，只要你有相应的权限，就可以用文件描述符来读写文件。从这个意义上说，文件描述符是一种**能力**（capability）[L84]，也就是一个不透明的句柄，赋予你执行某些操作的权力。理解文件描述符的另一种方式，是把它看作指向一个文件类型对象的指针；有了这样的对象，你就可以调用其他“方法”来访问文件，比如 read() 和 write()（下面会看到怎么做）。

如上所述，文件描述符由操作系统按进程管理。这意味着在 UNIX 系统中，proc 结构里保存着某种简单的结构（例如一个数组）。下面是 xv6 内核 [CK+08] 中的相关片段：

```
struct proc {
  ...
  struct file *ofile[NOFILE]; // 打开的文件
  ...
};
```

一个以文件描述符为下标的简单数组（最多可以有 NOFILE 个打开的文件），按进程记录打开了哪些文件。数组的每个条目其实只是一个指向 struct file 的指针，这个结构用来记录正在读写的文件的信息；下面会进一步讨论。

39.4 读写文件

有了文件，我们自然想要读写它们。先从读取一个已有的文件开始。如果是在命令行里操作，我们可能会直接用 cat 程序把文件内容输出到屏幕上。

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

在这段代码中，我们把 echo 程序的输出重定向（redirect）到文件 foo，于是这个文件里就有了单词“hello”。然后用 cat 查看文件的内容。但 cat 程序是怎么访问文件 foo 的呢?

提示：使用 strace（以及类似的工具）

strace 工具提供了一种很棒的方法，让你看清程序在做什么。运行它，你就能跟踪程序发出了哪些系统调用，看到参数和返回码，总之能把程序在干什么看得很清楚。

这个工具还接受一些很有用的参数。例如，-f 会同时跟踪 fork 出来的所有子进程（child process）；-t 会报告每次调用时的时刻；-e trace=open,close,read,write 只跟踪这几个系统调用，忽略其他所有调用。还有很多其他选项；去读读手册页（manual page），看看怎么用好这个出色的工具。

为了弄清这一点，我们要用一个极其有用的工具，来跟踪程序发出的系统调用。在 Linux 上，这个工具叫 **strace**；其他系统也有类似的工具（Mac 上有 **dtruss**，一些较老的 UNIX 变体上有 **truss**）。strace 的作用是跟踪程序运行期间发出的每一个系统调用，并把跟踪结果输出到屏幕上给你看。

下面是用 strace 弄清 cat 在做什么的一个例子（为了便于阅读，删掉了一些调用）：

```
prompt> strace cat foo
...
open("foo", O_RDONLY|O_LARGEFILE)       = 3
read(3, "hello\n", 4096)                = 6
write(1, "hello\n", 6)                  = 6
hello
read(3, "", 4096)                       = 0
close(3)                                = 0
...
prompt>
```

cat 做的第一件事是打开文件，准备读取。关于这一点，有几处值得注意：第一，文件只是以读的方式（而不是写）打开的，这由 O_RDONLY 标志表示；第二，使用的是 64 位偏移量（offset，O_LARGEFILE）；第三，open() 调用成功了，返回一个文件描述符，值为 3。

为什么第一次调用 open() 返回的是 3，而不是你可能以为的 0 或 1？原来，每个运行中的进程都已经打开了三个文件：标准输入（standard input），进程可以读它来获取输入；标准输出（standard output），进程可以写它来把信息输出到屏幕；标准错误（standard error），进程可以往里写错误信息。它们分别用文件描述符 0、1 和 2 表示。因此，第一次打开另一个文件时（就像上面的 cat 那样），得到的几乎肯定是文件描述符 3。

open 成功之后，cat 用 read() 系统调用反复从文件中读取一些字节。read() 的第一个参数是文件描述符，它告诉文件系统要读哪个文件；进程当然可以同时打开多个文件，所以操作系统要靠描述符才知道某次读操作针对的是哪个文件。第二个参数指向一个缓冲区（buffer），read() 的结果会放在这里；在上面的系统调用跟踪中，strace 在这个位置显示了读到的内容（“hello”）。第三个参数是缓冲区的大小，这里是 4 KB。read() 调用同样成功返回，返回值是读到的字节数（6，其中 5 个是单词“hello”的字母，还有 1 个是行尾标记）。

这时，你会在 strace 的输出中看到另一个有趣的结果：一次对文件描述符 1 的 write() 系统调用。前面提到过，这个描述符就是标准输出，因此这次调用把单词“hello”写到了屏幕上，这正是 cat 程序该做的事。但它是直接调用 write() 的吗？也许是（如果它做过高度优化）。如果不是，cat 可能会调用库例程 printf()；printf() 在内部处理好传给它的所有格式细节，最终写入标准输出，把结果打印到屏幕上。

接着，cat 程序试图从文件中读取更多内容，但文件里已经没有剩余的字节了，于是 read() 返回 0，程序就知道整个文件已经读完了。随后，程序调用 close() 并传入对应的文件描述符，表示它已经用完文件“foo”。文件就此关闭，读取也随之完成。

补充：数据结构——打开文件表
每个进程都维护着一个文件描述符数组，每个文件描述符都指向系统范围的**打开文件表**（open file table）中的一个条目。表中的每个条目记录着描述符对应的是哪个底层文件、当前偏移量（current offset），以及其他相关细节，比如文件是否可读、可写。

写文件也是类似的一系列步骤。首先以写的方式打开文件，然后调用 `write()` 系统调用（对较大的文件可能要调用多次），最后调用 `close()`。你可以用 `strace` 跟踪对文件的写操作，比如跟踪你自己写的程序，或者跟踪 `dd` 工具，例如 `dd if=foo of=bar`。

39.5 读写，但不按顺序

到目前为止，我们讨论了如何读写文件，但所有访问都是**顺序的**（sequential）；也就是说，我们要么从头到尾读一个文件，要么从头到尾写出一个文件。

不过，有时我们需要在文件的特定偏移量处读写；例如，如果你为一个文本文档建立了索引，并用它来查找某个特定的单词，最终可能要从文档中的某些**随机**（random）偏移量处读取。为此，我们要用到 `lseek()` 系统调用。下面是它的函数原型：

```
off_t lseek(int fildes, off_t offset, int whence);
```

第一个参数是大家熟悉的文件描述符。第二个参数是 `offset`，它把**文件偏移量**（file offset）定位到文件中的某个位置。第三个参数由于历史原因叫作 `whence`，它决定定位究竟如何进行。下面摘自手册页：

```
如果 whence 是 SEEK_SET，偏移量设为 offset 字节。
如果 whence 是 SEEK_CUR，偏移量设为当前位置加上
  offset 字节。
如果 whence 是 SEEK_END，偏移量设为文件大小加上
  offset 字节。
```

从这段描述可以看出，对于进程打开的每个文件，操作系统都会记录一个“当前”偏移量，它决定下一次读或写从文件的什么位置开始。因此，打开的文件这一抽象本身就带有一个当前偏移量，它有两种更新方式。第一种是读或写了 N 个字节时，当前偏移量加上 N；因此每次读写都会*隐式地*更新偏移量。第二种是用 `lseek` *显式地*更新，它按上面所说的方式修改偏移量。

你可能已经猜到，偏移量就保存在前面看到的那个 `struct file` 中，`struct proc` 引用的正是它。下面是 xv6 对这个结构的（简化）定义：

```
struct file {
  int ref;
  char readable;
  char writable;
  struct inode *ip;
  uint off;
};
```

补充：调用 lseek() 不会执行磁盘寻道

lseek() 这个系统调用的名字起得不好，许多学生想弄懂磁盘以及磁盘之上的文件系统是怎样工作的，却被它弄得困惑不已。不要把两者混为一谈！lseek() 调用只是修改操作系统内存中的一个变量，这个变量为某个特定进程记录它的下一次读或写从哪个偏移量开始。如果发给磁盘的读写请求和上一次不在同一条磁道（track）上，就得移动磁头（disk head），这才是磁盘寻道（seek）。更让人困惑的是，先用 lseek() 定位到文件中的随机位置，再在这些位置读写，确实会导致更多的磁盘寻道。因此，调用 lseek() 可能导致随后的读写发生寻道，但它本身绝对不会引起任何磁盘 I/O。

从这个结构可以看出，操作系统可以用它来判断打开的文件是可读还是可写（或者两者皆可）、它对应的是哪个底层文件（由 struct inode 指针 ip 指向），以及当前偏移量。还有一个引用计数（reference count，ref），我们会在下面进一步讨论。

这些 file 结构代表了系统中当前所有打开的文件；它们合起来，有时叫作**打开文件表**（open file table）。xv6 内核只是把它们保存为一个数组，整个表用一把锁（lock）保护：

```
struct {
  struct spinlock lock;
  struct file file[NFILE];
} ftable;
```

我们用几个例子把这一点讲得更清楚些。首先，跟踪一个进程：它打开一个文件（大小为 300 字节），然后反复调用 read() 系统调用读取这个文件，每次读 100 字节。下面是相关系统调用的跟踪结果，同时列出了每个系统调用的返回值，以及这次文件访问在打开文件表中的当前偏移量：

系统调用	返回值	当前偏移量
fd = open("file", O_RDONLY);	3	0
read(fd, buffer, 100);	100	100
read(fd, buffer, 100);	100	200
read(fd, buffer, 100);	100	300
read(fd, buffer, 100);	0	300
close(fd);	0	–

从这次跟踪中，有几点值得注意。首先，文件打开时，当前偏移量被初始化为零。其次，进程每调用一次 read()，偏移量都会增加；这样，进程只要不断调用 read()，就能读到文件的下一块内容。最后，试图越过文件末尾的 read() 返回零，以此告诉进程它已经把整个文件读完了。

第二个例子，跟踪一个进程：它把同一个文件打开两次，并对两者各发出一次读操作。

 © 2008–23, ARPACI-DUSSEAU

系统调用	返回值	OFT[10] 当前偏移量	OFT[11] 当前偏移量
`fd1 = open("file", O_RDONLY);`	3	0	–
`fd2 = open("file", O_RDONLY);`	4	0	0
`read(fd1, buffer1, 100);`	100	100	0
`read(fd2, buffer2, 100);`	100	100	100
`close(fd1);`	0	–	100
`close(fd2);`	0	–	–

在这个例子中，分配了两个文件描述符 3 和 4，它们各自指向打开文件表中不同的条目（在这个例子中是条目 10 和 11，如表头所示；OFT 表示 Open File Table，即打开文件表）。跟踪一下发生了什么，就能看到每个当前偏移量是如何独立更新的。

最后一个例子，进程在读取之前用 `lseek()` 重新定位当前偏移量；这种情况下只需要一个打开文件表条目（和第一个例子一样）。

系统调用	返回值	当前偏移量
`fd = open("file", O_RDONLY);`	3	0
`lseek(fd, 200, SEEK_SET);`	200	200
`read(fd, buffer, 50);`	50	250
`close(fd);`	0	–

在这里，`lseek()` 调用先把当前偏移量设为 200。随后的 `read()` 读取接下来的 50 个字节，并相应地更新当前偏移量。

39.6 共享文件表条目：`fork()` 和 `dup()`

在许多情况下（如上面的例子），文件描述符到打开文件表条目的映射是一对一的。例如，一个进程运行时，可能决定打开一个文件、读取它，然后关闭它；在这个例子里，这个文件在打开文件表中有一个唯一的条目。即使另一个进程同时读取同一个文件，每个进程也都会在打开文件表中有自己的条目。这样，每一次逻辑上的文件读写都是独立的，访问该文件时各有各的当前偏移量。

不过，也有几种有趣的情况，打开文件表中的条目是*共享*的。其中一种情况发生在父进程（parent process）用 `fork()` 创建子进程时。图 39.2 给出了一小段代码：父进程创建一个子进程，然后等待它完成。子进程调用 `lseek()` 调整当前偏移量，然后退出。最后，父进程等到子进程结束后，检查当前偏移量并把它的值打印出来。

运行这个程序，可以看到以下输出：

```
prompt> ./fork-seek
child: offset 10
parent: offset 10
prompt>
```

图 39.3 展示了三者之间的关系：每个进程私有的描述符数组、共享的打开文件表条目，以及这个条目对底层文件系统 inode 的引用。注意，这里我们终于用上了**引用计数**

```
int main(int argc, char *argv[]) {
    int fd = open("file.txt", O_RDONLY);
    assert(fd >= 0);
    int rc = fork();
    if (rc == 0) {
        rc = lseek(fd, 10, SEEK_SET);
        printf("child: offset %d\n", rc);
    } else if (rc > 0) {
        (void) wait(NULL);
        printf("parent: offset %d\n",
               (int) lseek(fd, 0, SEEK_CUR));
    }
    return 0;
}
```

图 39.2：**父子进程共享的文件表条目（`fork-seek.c`）**

（reference count）。文件表条目被共享时，引用计数会增加；只有两个进程都关闭了文件（或者都退出了），这个条目才会被删除。

在父子进程之间共享打开文件表条目偶尔很有用。例如，如果你创建了一些协同完成某项任务的进程，它们无须任何额外的协调就可以写同一个输出文件。关于调用 `fork()` 时进程之间共享了哪些东西，请查阅手册页。

另一种有趣的、也许更有用的共享情况，出现在 **`dup()`** 系统调用（以及它的近亲 **`dup2()`** 和 **`dup3()`**）中。

`dup()` 调用让进程创建一个新的文件描述符，它和一个已有的描述符指向同一个底层的打开文件。图 39.4 给出了一小段代码，展示 `dup()` 的用法。

`dup()` 调用（尤其是 `dup2()`）在编写 UNIX shell、执行输出重定向（output redirection）之类的操作时很有用；花点时间想想为什么！这时你可能在想：我做 shell 那个项目的时候，他们怎么不告诉我这个？唉，就算是一本了不起的操作系统书，也没法把所有东西都按正确的顺序讲。抱歉！

39.7 用 `fsync()` 立即写入

大多数时候，程序调用 `write()` 只是在告诉文件系统：请在将来的某个时候把这些数据写到持久存储上。出于性能考虑，文件系统会把这些写操作在内存中**缓冲**（buffer）一段时间（比如 5 秒或 30 秒）；到了那个稍后的时刻，写操作才会真正发给存储设备（storage device）。从调用方应用程序的角度看，写操作似乎很快就完成了，只有在极少数情况下（例如机器在 `write()` 调用之后、数据写入磁盘之前崩溃了）才会丢失数据。

然而，有些应用程序需要的不只是这种“最终会写入”的保证。例如，在数据库管理系统（DBMS）中，要开发正确的恢复协议（recovery protocol），就需要能够时不时地强制把数据写入磁盘。

为了支持这类应用程序，大多数文件系统都提供了一些额外的控制 API。在 UNIX 世界里，提供给应用程序的这个接口叫作 `fsync(int fd)`。进程对某个文件描述符调用

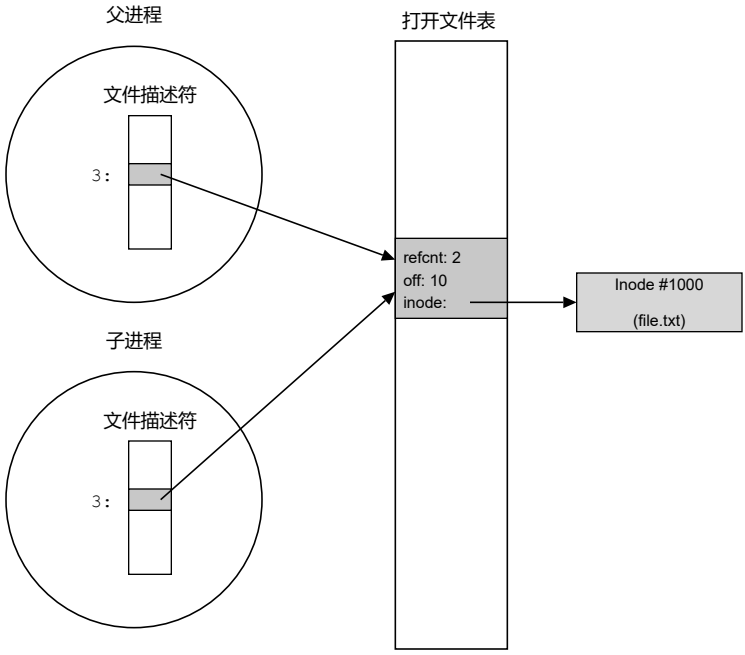


图 39.3：**共享同一个打开文件表条目的两个进程**

`fsync()` 时，文件系统会把该描述符所指文件的全部**脏**（dirty）数据（即还没写入的数据）强制写入磁盘。所有这些写操作完成后，`fsync()` 例程才返回。

下面是一个使用 `fsync()` 的简单例子。这段代码打开文件 `foo`，往里写入一块数据，然后调用 `fsync()`，确保这些写操作立即落到磁盘上。`fsync()` 一返回，应用程序就可以放心地继续执行，因为它知道数据已经持久化了（当然，前提是 `fsync()` 的实现是正确的）。

```
int main(int argc, char *argv[]) {
    int fd = open("README", O_RDONLY);
    assert(fd >= 0);
    int fd2 = dup(fd);
    // 现在 fd 和 fd2 可以互换使用
    return 0;
}
```

图 39.4：**使用 `dup()` 共享文件表条目（`dup.c`）**

```
int fd = open("foo", O_CREAT|O_WRONLY|O_TRUNC,
              S_IRUSR|S_IWUSR);
assert(fd > -1);
int rc = write(fd, buffer, size);
assert(rc == size);
rc = fsync(fd);
assert(rc == 0);
```

有意思的是，这个操作序列并不能保证你可能期望的一切；在某些情况下，你还需要对包含文件 foo 的目录调用 fsync()。加上这一步，不仅能确保文件本身已经写到磁盘上，还能确保这个文件（如果是新创建的）也已经持久地成为目录的一部分。毫不奇怪，这类细节常常被忽视，导致了许多应用层面的 bug [P+13, P+14]。

39.8 重命名文件

有了文件之后，有时需要给它换个名字。在命令行中，这可以用 mv 命令来完成；在这个例子中，文件 foo 被重命名为 bar：

```
prompt> mv foo bar
```

用 strace 可以看到，mv 使用的是系统调用 rename(char *old, char *new)，它正好接受两个参数：文件原来的名字（old）和新名字（new）。

rename() 调用提供了一个有意思的保证：它（通常）实现为一个**原子**（atomic）调用，即使遇到系统崩溃（system crash）也是如此；如果系统在重命名过程中崩溃，文件要么还叫旧名字，要么已经叫新名字，不会出现奇怪的中间状态。因此，对于某些需要原子地（atomically）更新文件状态的应用程序来说，rename() 至关重要。

我们说得再具体一点。假设你在用一个文件编辑器（例如 emacs），并在一个文件的中间插入了一行。在这个例子里，文件名是 foo.txt。编辑器为了保证新文件既有原来的内容、又有插入的那一行，可能会这样更新文件（为简单起见，省略了错误检查）：

```
int fd = open("foo.txt.tmp", O_WRONLY|O_CREAT|O_TRUNC,
              S_IRUSR|S_IWUSR);
write(fd, buffer, size); // 写出文件的新版本
fsync(fd);
close(fd);
rename("foo.txt.tmp", "foo.txt");
```

在这个例子中，编辑器做的事情很简单：用一个临时名字（foo.txt.tmp）写出文件的新版本，用 fsync() 把它强制写入磁盘，然后，等应用程序确定新文件的元数据（metadata）和内容都已经在磁盘上了，再把临时文件重命名为原来的文件名。最后这一步原子地把新文件换到原来的位置，同时删除文件的旧版本，从而实现了原子的文件更新。

39.9 获取文件信息

除了访问文件，我们还希望文件系统为它存储的每个文件保存相当多的信息。我们通常把这类关于文件的数据称为**元数据**（metadata）。要查看某个文件的元数据，可以使用

```
struct stat {
  dev_t     st_dev;     // 文件所在设备的 ID
  ino_t     st_ino;     // inode 号
  mode_t    st_mode;    // 保护（权限）
  nlink_t   st_nlink;   // 硬链接数
  uid_t     st_uid;     // 属主的用户 ID
  gid_t     st_gid;     // 属主的组 ID
  dev_t     st_rdev;    // 设备 ID（如果是特殊文件）
  off_t     st_size;    // 总大小，以字节计
  blksize_t st_blksize; // 文件系统 I/O 的块大小
  blkcnt_t  st_blocks;  // 已分配的块数
  time_t    st_atime;   // 最后访问时间
  time_t    st_mtime;   // 最后修改时间
  time_t    st_ctime;   // 最后状态变更时间
};
```

图 39.5：**stat 结构**

`stat()` 或 `fstat()` 系统调用。这两个调用接受一个文件的路径名（pathname）或文件描述符，并填充一个 `stat` 结构，如图 39.5 所示。

可以看到，每个文件都保存着大量信息，包括它的大小（以字节为单位）、low-level name（即 inode 号）、一些所有权信息，以及文件何时被访问或修改的一些信息，等等。要查看这些信息，可以使用命令行工具 `stat`。在下面的例子中，我们先创建一个文件（名叫 `file`），然后用命令行工具 `stat` 了解这个文件的一些情况。

下面是在 Linux 上的输出：

```
prompt> echo hello > file
prompt> stat file
  File: 'file'
  Size: 6       Blocks: 8      IO Block: 4096 regular file
Device: 811h/2065d Inode: 67158084  Links: 1
Access: (0640/-rw-r-----) Uid: (30686/remzi)
  Gid: (30686/remzi)
Access: 2011-05-03 15:50:20.157594748 -0500
Modify: 2011-05-03 15:50:20.157594748 -0500
Change: 2011-05-03 15:50:20.157594748 -0500
```

每个文件系统通常把这类信息保存在一个叫作 **inode**（索引节点）的结构中[1]。讲到文件系统的实现时，我们会深入学习 inode。现在，你只需把 inode 看作文件系统保存的一种持久数据结构（persistent data structure），里面存放着上面看到的那类信息。所有 inode 都驻留在磁盘上；活跃 inode 的副本通常会缓存（cache）在内存中，以加快访问速度。

[1]有些文件系统给这些结构起的名字相近但略有不同，比如 dnode；不过基本思想是类似的。

补充：mmap() 与持久内存

（客座补充，作者 Terence Kelly）

内存映射（memory mapping）是访问文件中持久数据的另一种方式。**mmap()** 系统调用在文件中的字节偏移量与调用进程中的虚拟地址（virtual address）之间建立对应关系；前者称为**后备文件**（backing file），后者称为它的**内存映像**（in-memory image）。之后，进程就可以对内存映像（in-memory image）执行 CPU 指令（即加载和存储），以此访问后备文件（backing file）。

把文件的持久性（persistence）和内存的访问语义结合起来，基于文件的内存映射就支持了一种叫作**持久内存**（persistent memory）的软件抽象。持久内存（persistent memory）这种编程风格不必在内存和存储的不同数据格式之间来回转换，因而能让应用程序更精简 [K19]。

```
1  p = mmap(NULL, file_size, PROT_READ|PROT_WRITE,
2           MAP_SHARED, fd, 0);
3  assert(p != MAP_FAILED);
4  for (int i = 1; i < argc; i++)
5    if (strcmp(argv[i], "pop") == 0) // 出栈
6      if (p->n > 0) // 栈不空
7        printf("%d\n", p->stack[--p->n]);
8    } else { // 入栈
9      if (sizeof(pstack_t) + (1 + p->n) * sizeof(int)
10         <= file_size) // 栈未满
11       p->stack[p->n++] = atoi(argv[i]);
12   }
```

程序 pstack.c（收录在 OSTEP 代码的 GitHub 仓库中，上面是其中的一个片段）在文件 ps.img 中保存了一个持久的栈（stack）；这个文件起初只是一堆零，比如可以在命令行中用 truncate 或 dd 工具创建。文件里包含栈的大小计数，以及保存栈内容的整数数组。用 mmap() 映射后备文件（backing file）之后，我们就可以通过指向内存映像（in-memory image）的 C 指针来访问这个栈，例如用 p->n 访问栈中元素的个数，用 p->stack 访问整数数组。由于栈是持久的，一次运行 pstack 时 push 进去的数据，下一次运行时可以 pop 出来。

如果发生崩溃，比如在 push 的自增和赋值之间崩溃，持久栈就可能处于不一致的状态。应用程序可以借助一些机制来防止这类损坏：这些机制更新持久内存（persistent memory）时，即使中途发生故障也能保证原子性（atomicity）[K20]。

39.10 删除文件

到这里，我们已经知道如何创建文件，以及如何顺序或不按顺序地访问文件。但怎么删除文件呢？如果你用过 UNIX，可能会觉得自己知道：运行 rm 程序就行。但 rm 是用哪个系统调用来删除文件的呢？

我们再请出老朋友 strace 来找答案。这里我们删除那个烦人的文件 foo：

```
prompt> strace rm foo
...
unlink("foo")                           = 0
...
```

我们从跟踪输出中删掉了一大堆无关的内容，只留下一个对 `unlink()` 的调用，这个系统调用的名字很神秘。可以看到，`unlink()` 只接受要删除的文件的名字，成功时返回零。但这就引出了一个大谜团：这个系统调用为什么叫 `unlink`？为什么不直接叫 `remove` 或 `delete`？

要解开这个谜，我们不仅要了解文件，还得了解目录。

39.11 创建目录

除了文件，还有一组与目录相关的系统调用，让你可以创建、读取和删除目录。注意，你永远不能直接写目录：目录的格式被视为文件系统的元数据，文件系统认为自己要对目录数据的完整性（integrity）负责；因此，你只能间接地更新目录，比如在目录中创建文件、目录或其他类型的对象。文件系统靠这种办法确保目录的内容符合预期。

要创建目录，可以使用系统调用 `mkdir()`。同名的 `mkdir` 程序可以用来创建这样的目录。我们来看看运行 `mkdir` 程序创建一个名为 `foo` 的简单目录时会发生什么：

```
prompt> strace mkdir foo
...
mkdir("foo", 0777)                      = 0
...
prompt>
```

这样创建的目录虽然有最基本的一点内容，仍算是“空”的。具体来说，空目录（empty directory）有两个条目：一个条目指向它自己，另一个条目指向它的父目录。前者叫作“.”（dot）目录，后者叫作“..”（dot-dot）目录。给 `ls` 程序传一个选项（`-a`），就能看到这两个目录：

```
prompt> ls -a
./  ../
prompt> ls -al
total 8
drwxr-x---  2 remzi remzi    6 Apr 30 16:17 ./
drwxr-x--- 26 remzi remzi 4096 Apr 30 16:17 ../
```

39.12 读取目录

既然已经创建了目录，我们可能也想读取目录。其实，`ls` 程序做的正是这件事。我们来写一个类似 `ls` 的小工具，看看它是怎么做到的。

我们不是像打开文件那样直接打开目录，而是使用一组新的调用。下面的示例程序打印一个目录的内容。这个程序用了三个调用：`opendir()`、`readdir()` 和 `closedir()`，

提示：小心威力强大的命令

rm 程序是一个很好的例子，说明了什么是威力强大的命令，以及威力太大有时也是坏事。例如，要一次删除一批文件，你可以输入类似这样的命令：

```
prompt> rm *
```

其中 * 会匹配当前目录（current directory）中的所有文件。但有时候你还想把目录也删掉，而且是连同目录里的全部内容一起删掉。办法是让 rm 递归地进入每个目录，把里面的内容也删掉：

```
prompt> rm -rf *
```

如果你不小心在文件系统的根目录下执行了这一小串字符，麻烦就来了：它会把文件系统中的每个文件和目录都删掉。哎呀！

所以，要记住威力强大的命令是一把双刃剑：它们让你只敲几下键盘就能完成大量工作，但也能迅速而轻易地造成巨大的破坏。

你可以看到这个接口有多简单；我们只用一个简单的循环，每次读取一个目录项（directory entry），打印出目录中每个文件的名字和 inode 号。

```
int main(int argc, char *argv[]) {
    DIR *dp = opendir(".");
    assert(dp != NULL);
    struct dirent *d;
    while ((d = readdir(dp)) != NULL) {
        printf("%lu %s\n", (unsigned long) d->d_ino,
               d->d_name);
    }
    closedir(dp);
    return 0;
}
```

从下面的声明可以看到，struct dirent 中的每个目录项（directory entry）都包含哪些信息：

```
struct dirent {
  char           d_name[256]; // 文件名
  ino_t          d_ino;       // inode 号
  off_t          d_off;       // 到下一个 dirent 的偏移量
  unsigned short d_reclen;    // 本记录的长度
  unsigned char  d_type;      // 文件类型
};
```

由于目录里的信息很少，基本上只是把名字映射到 inode 号，外加少量其他细节，程序可能还想对每个文件调用 stat()，获取它的长度等更多信息。给 ls 传入 -l 选项时，它正是这么做的；你可以分别带上和不带这个选项，用 strace 跟踪 ls，亲眼看看。

39.13 删除目录

最后，可以调用 `rmdir()` 来删除目录，同名的 `rmdir` 程序用的就是它。不过，删除目录比删除文件更危险，因为一条命令就可能删掉大量数据。因此，`rmdir()` 要求目录在删除之前必须是空的（即只有“.”和“..”两个条目）。如果你试图删除一个非空目录（non-empty directory），`rmdir()` 调用就会直接失败。

39.14 硬链接

现在我们回到那个谜：为什么删除文件用的是 `unlink()`。为此，我们要了解在文件系统树中创建条目的一种新方法，也就是一个叫作 `link()` 的系统调用。`link()` 系统调用接受两个参数：一个旧路径名和一个新路径名；把一个新文件名“链接”到一个旧文件名上，实际上就是创建了引用同一个文件的另一种方式。命令行程序 `ln` 就是用来做这件事的，如下例所示：

```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2
prompt> cat file2
hello
```

这里我们创建了一个文件，里面有单词“hello”，并把这个文件命名为 `file`[2]。然后，我们用 `ln` 程序为这个文件创建了一个硬链接（hard link）。之后，打开 `file` 或 `file2` 都能查看这个文件。

`link()` 的工作方式是：它只是在你创建链接的那个目录中再创建一个名字，并让它指向原文件的同一个 inode 号。文件本身根本没有复制；你现在只是有了两个人类可读的名字（`file` 和 `file2`），它们都指向同一个文件。我们甚至可以在目录本身中看到这一点，只要打印出每个文件的 inode 号：

```
prompt> ls -i file file2
67158084 file
67158084 file2
prompt>
```

给 `ls` 传入 `-i` 选项，它就会打印出每个文件的 inode 号以及文件名。由此可以看出 `link` 到底做了什么：只是为同一个 inode 号（这个例子中是 67158084）新建了一个引用。

现在你也许开始明白 `unlink()` 为什么叫 `unlink()` 了。创建一个文件时，你其实在做两件事。第一，你在创建一个结构（inode），它将记录这个文件几乎所有的相关信息，包括它的大小、它的块（block）在磁盘上的位置等等。第二，你在把一个人类可读的名字链接到这个文件上，并把这个链接放进一个目录中。

[2]再次注意本书作者有多么富有创意。我们以前还养过一只叫“Cat”的猫（真事）。不过，她死了，现在我们养了一只叫“Hammy”的仓鼠。更新：Hammy 现在也死了。宠物的尸体越堆越多了。

为文件创建硬链接之后，在文件系统看来，原来的文件名（file）和新建的文件名（file2）没有任何区别：两者都只是指向文件底层元数据的链接，这些元数据保存在编号为 67158084 的 inode 中。

因此，要从文件系统中删除一个文件，我们调用 unlink()。比如在上面的例子中，我们可以删除名为 file 的文件，之后仍然可以毫无困难地访问这个文件：

```
prompt> rm file
removed 'file'
prompt> cat file2
hello
```

之所以能这样，是因为文件系统对 file 执行 unlink 时，会检查 inode 中的一个**引用计数**（reference count）。这个引用计数（有时也叫**链接计数**，link count）让文件系统可以记录有多少个不同的文件名链接到了这个 inode 上。调用 unlink() 时，它会删除人类可读的名字（正在被删除的文件）与给定 inode 号之间的“链接”，并把引用计数减一；只有引用计数降到零，文件系统才会同时释放 inode 及相关的数据块（data block），从而真正“删除”这个文件。

当然，你可以用 stat() 查看文件的引用计数。我们来看看创建和删除文件的硬链接时，它是怎么变化的。在这个例子中，我们为同一个文件创建三个链接，然后再把它们删掉。注意看链接计数！

```
prompt> echo hello > file
prompt> stat file
... Inode: 67158084     Links: 1 ...
prompt> ln file file2
prompt> stat file
... Inode: 67158084     Links: 2 ...
prompt> stat file2
... Inode: 67158084     Links: 2 ...
prompt> ln file2 file3
prompt> stat file
... Inode: 67158084     Links: 3 ...
prompt> rm file
prompt> stat file2
... Inode: 67158084     Links: 2 ...
prompt> rm file2
prompt> stat file3
... Inode: 67158084     Links: 1 ...
prompt> rm file3
```

39.15 符号链接

还有一种链接非常有用，叫作**符号链接**（symbolic link），有时也叫**软链接**（soft link）。硬链接有一些局限：不能为目录创建硬链接（以免在目录树中造成环）；不能硬链接到其他磁盘分区中的文件（因为 inode 号只在某个文件系统内唯一，跨文件系统就不唯一了）；等等。因此，人们发明了一种叫作符号链接的新型链接 [MJLF84]。

要创建这种链接，可以使用同一个程序 `ln`，但要加上 `-s` 选项。下面是一个例子：

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
```

可以看到，创建软链接看上去差不多，现在既可以通过文件名 `file`，也可以通过符号链接名 `file2` 访问原来的文件。

不过，除了表面上的相似之处，符号链接其实和硬链接很不一样。第一个区别是，符号链接本身其实是一个文件，只是类型不同。我们已经讨论过普通文件和目录；符号链接是文件系统所知道的第三种类型。对符号链接执行 `stat` 就能看出一切：

```
prompt> stat file
... regular file ...
prompt> stat file2
... symbolic link ...
```

运行 `ls` 也能看出这一点。如果仔细看 `ls` 的长格式输出，会发现最左边一列的第一个字符：普通文件是 `-`，目录是 `d`，软链接是 `l`。还可以看到符号链接的大小（这里是 4 字节），以及链接指向的东西（名为 `file` 的文件）。

```
prompt> ls -al
drwxr-x---  2 remzi remzi   29 May  3 19:10 ./
drwxr-x--- 27 remzi remzi 4096 May  3 15:14 ../
-rw-r-----  1 remzi remzi    6 May  3 19:10 file
lrwxrwxrwx  1 remzi remzi    4 May  3 19:10 file2 -> file
```

`file2` 之所以是 4 字节，是因为符号链接的做法，就是把被链接文件的路径名保存为链接文件的数据。由于我们链接到的是一个名为 `file` 的文件，链接文件 `file2` 就很小（4 字节）。如果链接到一个更长的路径名，链接文件就会更大：

```
prompt> echo hello > alongerfilename
prompt> ln -s alongerfilename file3
prompt> ls -al alongerfilename file3
-rw-r----- 1 remzi remzi  6 May  3 19:17 alongerfilename
lrwxrwxrwx 1 remzi remzi 15 May  3 19:17 file3 ->
alongerfilename
```

最后，由于符号链接的创建方式，它们可能会产生所谓的**悬空引用**（dangling reference）：

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
prompt> rm file
prompt> cat file2
cat: file2: No such file or directory
```

从这个例子可以看出，和硬链接很不一样，删除名为 `file` 的原文件后，链接指向的就是一个不再存在的路径名了。

39.16 权限位和访问控制列表

进程这一抽象提供了两种核心的虚拟化：CPU 的虚拟化和内存的虚拟化。它们分别给进程营造出一种假象：进程拥有自己*私有的* CPU 和自己*私有的*内存；而实际上，底层的操作系统用各种技术，在相互竞争的实体之间安全可靠地共享有限的物理资源。

正如本章所描述的，文件系统也呈现了磁盘的一种虚拟视图，把它从一堆原始的块变成了用起来友好得多的文件和目录。不过，这个抽象和 CPU、内存的抽象有一个显著的不同：文件通常在不同的用户和进程之间*共享*，而不（总）是私有的。因此，文件系统中通常有一套更全面的机制，用来支持不同程度的共享。

这类机制的第一种形式，是经典的 UNIX **权限位**（permission bits）。要查看文件 `foo.txt` 的权限，只需输入：

```
prompt> ls -l foo.txt
-rw-r--r--  1 remzi  wheel  0 Aug 24 16:29 foo.txt
```

我们只关注这段输出的第一部分，也就是 `-rw-r--r--`。这里的第一个字符只表示文件的类型：`-` 表示普通文件（`foo.txt` 就是），`d` 表示目录，`l` 表示符号链接，等等；这（大体上）和权限无关，所以我们暂时忽略它。

我们感兴趣的是权限位，由接下来的九个字符表示（`rw-r--r--`）。这些位决定了每个普通文件、目录和其他实体究竟谁能访问、能怎样访问。

权限分为三组：文件的**属主**（owner）可以对它做什么，某个**组**（group）中的人可以对它做什么，最后是其他任何人（有时称为**其他用户**，other）可以做什么。属主、组成员或其他用户可以拥有的能力包括读文件、写文件和执行文件。

在上面的例子中，`ls` 输出的前三个字符表明，这个文件属主可读可写（`rw-`），`wheel` 组的成员以及系统中的其他任何人则只能读（`r--` 后面再跟 `r--`）。

文件的属主可以很方便地修改这些权限，比如使用 `chmod` 命令（用来修改**文件模式**，file mode）。要去掉除属主以外所有人访问这个文件的能力，可以输入：

```
prompt> chmod 600 foo.txt
```

这条命令为属主打开了可读位（4）和可写位（2）（两者按位或得到上面的 6），而把组和其他用户的权限位分别设为 0 和 0，从而把权限设为 `rw-------`。

执行位尤其有意思。对于普通文件，它决定了程序能不能运行。例如，假设有一个名为 `hello.csh` 的简单 shell 脚本，我们可能想输入以下命令来运行它：

```
prompt> ./hello.csh
hello, from shell world.
```

但是，如果没有为这个文件正确设置执行位，就会发生下面的事：

补充：文件系统的超级用户

哪个用户可以执行特权操作（privileged operation）来帮助管理文件系统？例如，如果需要删除某个不活跃用户的文件来节省空间，谁有权这么做？

在本地文件系统上，常见的默认做法是设立某种**超级用户**（superuser，即 **root**），它不论权限如何都可以访问所有文件。在 Andrew 文件系统（Andrew File System，AFS）这样带有访问控制列表（access control list）的分布式文件系统（distributed file system）中，名为 `system:administrators` 的组里是受信任、可以做这些事的用户。无论哪种情况，这些受信任的用户本身就是一种安全风险；如果攻击者能够设法冒充这样的用户，就能访问系统中的所有信息，从而破坏预期的隐私和保护保证。

```
prompt> chmod 600 hello.csh
prompt> ./hello.csh
./hello.csh: Permission denied.
```

对于目录，执行位的作用略有不同。具体来说，它让用户（或组，或所有人）能够进入这个目录（即用 `cd` 切换过去），并且在同时具有可写位时，能够在其中创建文件。要更多地了解这一点，最好的办法是：自己动手试试！别担心，你（多半）不会把什么东西搞得太糟。

除了权限位，一些文件系统，比如名为 AFS 的分布式文件系统（后面的章节会讨论），还提供了更精细的控制。例如，AFS 为每个目录设一个**访问控制列表**（access control list，**ACL**）来实现这一点。访问控制列表是一种更通用、更强大的方式，能够准确地表示谁可以访问某个资源。在文件系统中，它让用户能够列出一份非常具体的名单，指明谁能读、谁不能读一组文件；相比之下，上面描述的权限位那种“属主/组/所有人”模型就比较受限了。

例如，下面是一位作者的 AFS 账户中一个私有目录的访问控制（access control），由 `fs listacl` 命令显示：

```
prompt> fs listacl private
Access list for private is
Normal rights:
  system:administrators rlidwka
  remzi rlidwka
```

这份列表表明，系统管理员和用户 `remzi` 都可以在这个目录中查找（lookup）、插入、删除和管理文件，还可以读、写和锁定这些文件。

要允许某个人（这里是另一位作者）访问这个目录，用户 `remzi` 只需输入下面的命令：

```
prompt> fs setacl private/ andrea rl
```

`remzi` 的隐私就这样没了！不过你现在学到了一个更重要的道理：好的婚姻里不能有秘密，即使在文件系统里也一样[3]。

[3]如果你好奇的话：我们从 1996 年起就幸福地结了婚。我们知道，你并不好奇。

39.17 创建和挂载文件系统

我们已经浏览了访问文件、目录以及某些特殊类型链接的基本接口。但还有一个话题需要讨论：如何用许多底层文件系统组装出一棵完整的目录树。做法是：先创建文件系统，然后挂载（mount）它们，让它们的内容可以访问。

要创建文件系统，大多数文件系统都提供一个工具，通常叫作 `mkfs`（读作“make fs”），专门完成这项任务。思路是这样的：把一个设备（比如磁盘分区，例如 `/dev/sda1`）和一种文件系统类型（例如 ext3）作为输入交给这个工具，它就会在这个磁盘分区上写入一个空的文件系统，从根目录开始。于是 mkfs 说，要有文件系统！

然而，文件系统创建出来以后，还得让它能在统一的文件系统树中访问到。这项任务由 `mount` 程序完成（实际工作由底层的系统调用 `mount()` 来做）。`mount` 做的事情很简单：把一个已有的目录作为目标**挂载点**（mount point），实际上就是把一个新的文件系统贴到目录树的这个位置上。

举个例子可能更好理解。假设我们有一个还没挂载的 ext3 文件系统，存放在设备分区 `/dev/sda1` 中，它的内容如下：一个根目录，其中包含两个子目录 `a` 和 `b`，每个子目录中又各有一个名为 `foo` 的文件。假设我们想把这个文件系统挂载到挂载点（mount point）`/home/users`。我们会输入类似这样的命令：

```
prompt> mount -t ext3 /dev/sda1 /home/users
```

如果成功，`mount` 就会让这个新的文件系统变得可用。不过，要注意现在访问这个新文件系统的方式。要查看它根目录的内容，我们要这样使用 `ls`：

```
prompt> ls /home/users/
a b
```

可以看到，路径名 `/home/users/` 现在指向的是新挂载的这个目录的根。同样，我们可以用路径名 `/home/users/a` 和 `/home/users/b` 访问目录 `a` 和 `b`。最后，名为 `foo` 的两个文件可以通过 `/home/users/a/foo` 和 `/home/users/b/foo` 访问。这就是 mount 的妙处：它没有让许多文件系统各自分开，而是把所有文件系统统一到一棵树中，让命名变得统一而方便。

要查看系统上挂载了哪些文件系统、挂载在哪里，只需运行 `mount` 程序。你会看到类似这样的输出：

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/sda5 on /tmp type ext3 (rw)
/dev/sda7 on /var/vice/cache type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
AFS on /afs type afs (rw)
```

这个乱七八糟的组合表明，许多不同的文件系统都被粘合到了这台机器的文件系统树上，包括 ext3（一种标准的基于磁盘的文件系统）、proc 文件系统（一个用于访问当前进程

提示：小心 TOCTTOU

1974 年，McPhee 注意到计算机系统中的一个问题。具体来说，McPhee 指出："……如果在有效性检查与该有效性检查所关联的操作之间存在一段时间间隔，[并且]通过多任务处理，有效性检查所用的变量可以在这段时间间隔内被蓄意更改，就会导致控制程序执行一个无效的操作。"今天我们把这称为 **检查时刻到使用时刻**（Time Of Check To Time Of Use，**TOCTTOU**）问题，可惜的是，它至今仍可能发生。

Bishop 和 Dilger [BD96] 描述了一个简单的例子，展示用户如何欺骗一个更受信任的服务，从而引发麻烦。例如，设想一个邮件服务以 root 身份运行（因此有权访问系统上的所有文件）。这个服务按如下方式把收到的消息追加到用户的收件箱文件中。首先，它调用 `lstat()` 获取这个文件的信息，特别是确认它确实只是一个属于目标用户的普通文件，而不是指向其他文件的链接（邮件服务器不该更新那些文件）。然后，检查通过之后，服务器用新消息更新这个文件。

不幸的是，检查和更新之间的间隙会导致一个问题：攻击者（在这个例子里，就是收信的那个用户，因此有权访问收件箱）调用 `rename()`，把收件箱文件换成指向一个敏感文件，比如 `/etc/passwd`（它保存着用户及其密码的信息）。如果这个替换恰好发生在合适的时机（在检查和访问之间），服务器就会若无其事地用邮件内容更新这个敏感文件。现在，攻击者只需发一封电子邮件就能写这个敏感文件，这是一种权限提升（privilege escalation）；通过更新 `/etc/passwd`，攻击者可以添加一个具有 root 权限的账户，从而控制整个系统。

TOCTTOU 问题没有什么简单又出色的解决办法 [T+08]。一种方法是减少需要 root 权限才能运行的服务的数量，这会有所帮助。`O_NOFOLLOW` 标志能让 `open()` 在目标是符号链接时失败，从而避免那些需要借助符号链接的攻击。更激进的方法，比如使用**事务文件系统**（transactional file system）[H+18]，可以解决这个问题，但广泛部署的事务文件系统并不多。所以，只能给出那条老生常谈（也挺无力）的建议：编写以高权限运行的代码时要小心！

信息的文件系统）、tmpfs（一个专门存放临时文件的文件系统）和 AFS（一个分布式文件系统，distributed file system）。

39.18 小结

UNIX 系统（其实任何系统都是这样）的文件系统接口看上去相当简陋，但如果想掌握它，要理解的东西还很多。当然，没有什么比直接（大量地）使用它更好的了。所以请多用吧！当然，也要多读些资料；和往常一样，可以先从 Stevens 的书 [SR05] 读起。

补充：文件系统的关键术语

- **文件**是一个字节数组，可以创建、读取、写入和删除。它有一个唯一指代它的 low-level name（即一个数字）。这个 low-level name 通常叫作 **i-number**。
- **目录**（directory）是元组的集合，每个元组包含一个人类可读的名字，以及它所映射的 low-level name。每个条目要么指向另一个目录，要么指向一个文件。每个目录本身也有一个 low-level name（i-number）。目录总有两个特殊条目：. 条目指向目录自己，.. 条目指向它的父目录。
- **目录树**（directory tree）或**目录层次结构**（directory hierarchy）把所有文件和目录组织成一棵大树，从**根**开始。
- 要访问文件，进程必须用一个系统调用（通常是 `open()`）向操作系统请求权限。如果获得许可，操作系统就返回一个**文件描述符**（file descriptor），之后可以在权限和意图允许的范围内，用它来读或写。
- 每个文件描述符都是一个进程私有的实体，它指向**打开文件表**（open file table）中的一个条目。这个条目记录了此次访问针对的是哪个文件、文件的**当前偏移量**（current offset，即下一次读或写将访问文件的哪一部分），以及其他相关信息。
- 调用 `read()` 和 `write()` 自然会更新当前偏移量；此外，进程可以用 `lseek()` 修改它的值，从而能够随机访问（random access）文件的不同部分。
- 要把更新强制写入持久介质，进程必须使用 `fsync()` 或相关的调用。不过，要在保持高性能的同时正确地做到这一点很有挑战性 [P+14]，所以这样做时要仔细考虑。
- 要让文件系统中的多个人类可读的名字指向同一个底层文件，可以使用**硬链接**（hard link）或**符号链接**（symbolic link）。两者在不同的情况下各有用处，所以使用前要考虑它们各自的优缺点。还要记住，删除文件只不过是把它从目录层次结构（directory hierarchy）中最后一次 `unlink()` 掉而已。
- 大多数文件系统都有启用和禁止共享的机制。**权限位**（permission bits）提供了这类控制的一种简单形式；更精细的**访问控制列表**（access control list）可以更精确地控制谁能访问和操作信息。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[BD96] "Checking for Race Conditions in File Accesses" by Matt Bishop, Michael Dilger. Computing Systems 9:2, 1996. *A great description of the TOCTTOU problem and its presence in file systems.*

[CK+08] "The xv6 Operating System" by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. From: https://github.com/mit-pdos/xv6-public. *As mentioned before, a cool and simple Unix implementation. We have been using an older version (2012-01-30-1-g1c41342) and hence some examples in the book may not match the latest in the source.*

[H+18] "TxFS: Leveraging File-System Crash Consistency to Provide ACID Transactions" by Y. Hu, Z. Zhu, I. Neal, Y. Kwon, T. Cheng, V. Chidambaram, E. Witchel. USENIX ATC '18, June 2018. *The best paper at USENIX ATC '18, and a good recent place to start to learn about transactional file systems.*

[K19] "Persistent Memory Programming on Conventional Hardware" by Terence Kelly. ACM Queue, 17:4, July/August 2019. *A great overview of persistent memory programming; check it out!*

[K20] "Is Persistent Memory Persistent?" by Terence Kelly. Communications of the ACM, 63:9, September 2020. *An engaging article about how to test hardware failures in system on the cheaps; who knew breaking things could be so fun?*

[K84] "Processes as Files" by Tom J. Killian. USENIX, June 1984. *The paper that introduced the /proc file system, where each process can be treated as a file within a pseudo file system. A clever idea that you can still see in modern* UNIX *systems.*

[L84] "Capability-Based Computer Systems" by Henry M. Levy. Digital Press, 1984. Available: http://homes.cs.washington.edu/~levy/capabook. *An excellent overview of early capability-based systems.*

[MJLF84] "A Fast File System for UNIX" by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM TOCS, 2:3, August 1984. *We'll talk about the Fast File System (FFS) explicitly later on. Here, we refer to it because of all the other random fun things it introduced, like long file names and symbolic links. Sometimes, when you are building a system to improve one thing, you improve a lot of other things along the way.*

[P+13] "Towards Efficient, Portable Application-Level Consistency" by Thanumalayan S. Pillai, Vijay Chidambaram, Joo-Young Hwang, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. HotDep '13, November 2013. *Our own work that shows how readily applications can make mistakes in committing data to disk; in particular, assumptions about the file system creep into applications and thus make the applications work correctly only if they are running on a specific file system.*

[P+14] "All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications" by Thanumalayan S. Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. OSDI '14, Broomfield, Colorado, October 2014. *The full conference paper on this topic – with many more details and interesting tidbits than the first workshop paper above.*

[SK09] "Principles of Computer System Design" by Jerome H. Saltzer and M. Frans Kaashoek. Morgan-Kaufmann, 2009. *This tour de force of systems is a must-read for anybody interested in the field. It's how they teach systems at MIT. Read it once, and then read it a few more times to let it all soak in.*

[SR05] "Advanced Programming in the UNIX Environment" by W. Richard Stevens and Stephen A. Rago. Addison-Wesley, 2005. *We have probably referenced this book a few hundred thousand times. It is that useful to you, if you care to become an awesome systems programmer.*

[T+08] "Portably Solving File TOCTTOU Races with Hardness Amplification" by D. Tsafrir, T. Hertz, D. Wagner, D. Da Silva. FAST '08, San Jose, California, 2008. *Not the paper that introduced TOCTTOU, but a recent-ish and well-done description of the problem and a way to solve the problem in a portable manner.*

作业（编程）

在这个作业中，我们只是熟悉一下本章介绍的 API 是如何工作的。为此，你只需编写几个不同的程序，它们大多基于各种 UNIX 工具。

问题

1. **Stat：**编写你自己版本的命令行程序 `stat`，它只是对给定的文件或目录调用 `stat()` 系统调用。打印出文件大小、已分配的块数、引用（链接）计数等等。随着目录中条目数的变化，目录的链接计数是多少？有用的接口：当然是 `stat()`。
2. **列出文件：**编写一个程序，列出给定目录中的文件。不带任何参数调用时，程序应该只打印文件名。带 `-l` 选项调用时，程序应该打印出每个文件的信息，比如属主、组、权限，以及从 `stat()` 系统调用获得的其他信息。程序应该再接受一个参数，即要读取的目录，例如 `myls -l directory`。如果没有给出目录，程序就应该使用当前工作目录（current working directory）。有用的接口：`stat()`、`opendir()`、`readdir()`、`getcwd()`。
3. **Tail：**编写一个程序，打印出文件的最后几行。程序要高效：它应该先定位到文件末尾附近，读入一块数据，然后往回找，直到找到所要求的行数；这时，它应该从这些行的开头一直打印到文件末尾。调用这个程序时，应该输入 `mytail -n file`，其中 `n` 是要打印的文件末尾的行数。有用的接口：`stat()`、`lseek()`、`open()`、`read()`、`close()`。
4. **递归搜索：**编写一个程序，从文件系统树中的某个给定位置开始，打印出树中每个文件和目录的名字。例如，不带参数运行时，程序应该从当前工作目录（current working directory）开始，打印它的内容以及所有子目录的内容等等，直到把以当前工作目录（CWD）为根的整棵树都打印出来。如果给出一个参数，即目录名，就改用它作为树的根。再给你的递归搜索加上一些更有趣的选项，类似于强大的命令行工具 `find`。有用的接口：你自己去找吧。

 © 2008–23, ARPACI-DUSSEAU